

Слайд 1: Титульный. История и авторы

Я бы хотел рассказать об одной из **исторически важных** структур данных в программировании — об AVL-дереве. Это первая в мире структура динамически сбалансированного дерева поиска. Она была разработана ещё в 1962 году, и не абы кем, а советскими математиками Георгием Адельсон-Вельским и Евгением Ландисом. Первые буквы фамилий этих учёных и легли в аббревиатуру АВЛ. До изобретения этой структуры данных программисты сталкивались с серьёзной проблемой: деревья поиска со временем теряли свою эффективность и не могли гарантировать быстрое действие при добавлении новых данных. AVL-дерево стало первым и весьма производительным математическим решением этой проблемы.

Слайд 2: Проблема обычных деревьев поиска

Чтобы понять ценность изобретения, давайте посмотрим на классическое бинарное дерево. В теории оно обещает нам быстрый поиск за $O(\log n)$. Но на практике есть критический изъян. Если мы начнем добавлять в него уже отсортированные данные — например, последовательность чисел 1, 2, 3, 4, 5 — наше дерево выродится. Оно превратится в обычный двусвязный список. Вся древовидная структура исчезает, и алгоритмическая сложность падает до линейной, то есть до $O(n)$. Алгоритм будет вынужден перебирать каждый элемент по очереди. AVL-дерево было создано именно для того, чтобы архитектурно запретить дереву превращаться в список, независимо от того, в каком порядке поступают данные.

Слайд 3: Принцип баланса и показатель

Как же алгоритм понимает, что дерево начало искривляться? Вводится ключевое понятие — *Balance Factor*, или фактор баланса. Для абсолютно каждого узла в дереве мы вычисляем разницу между высотой его левого и правого поддеревьев. $BF(v) == h(\text{left}) - h(\text{right})$. В AVL-дереве введено жесткое правило: этот показатель может быть равен только -1, 0 или 1 (проще говоря, не превосходить единицу по модулю). Если разница высот достигает двух, значит дерево начало становиться несбалансированным. AVL-дерево понимает, что одна из ветвей стала слишком тяжелой, и немедленно запускает процедуру балансировки, чтобы вернуть показатель в норму.

Слайд 4: Механика: Типы поворотов

Процедура балансировки заключается в выполнении поворотов поддеревьев. Существует четыре типа таких операций. Если перевес возник на внешней стороне дерева, применяются одинарные повороты — левый или правый. Если ветка искривилась внутрь, образуя “колени”, применяются двойные повороты — сначала ветка выпрямляется в одну сторону, а затем поворачивается в другую. Наверное здесь стоит остановиться подробнее...

- **Правый поворот** применяется, когда **левый потомок** некоторого узла y (назовём его x) имеет перевес в своём **левом поддереве**.

Он работает следующим образом:

1. Узел x становится новым корнем поддерева вместо y .
2. Бывшее правое поддерево узла x становится левым поддеревом узла y .
3. Узел y становится правым потомком узла x .

- **Левый поворот** применяется, когда **правый потомок** узла перевешивает из-за своего **правого поддерева**. Он работает аналогично правому, но все действия происходят зеркально.

То есть:

1. Узел x становится новым корнем поддерева.
2. Левое поддерево узла x становится правым поддеревом узла y .
3. Узел y становится левым потомком x .

- **Лево-правый поворот** выполняется, когда у узла y **левый потомок** x перевешивает из-за своего **правого поддерева**.

В таком случае мы сначала выполняем левый поворот вокруг узла x , после чего это поддерево примет форму, как перед правым поворотом в описанном выше случае. И затем мы выполняем правый поворот вокруг узла y , приводя дерево к сбалансированному виду.

- **Право-левый поворот** — у узла y **правый потомок** x перевешивает из-за своего **левого поддерева**.

Сначала выполняем правый поворот вокруг x , а затем левый поворот вокруг y .

Звучит сложно, но на уровне кода это реализуется всего лишь переназначением нескольких указателей в памяти. Нам не нужно перемещать сами данные или копировать узлы. Поэтому любой поворот выполняется за строго константное время $O(1)$.

Слайд 5: Разбор примера (Вставка элемента 32)

Давайте посмотрим на пошаговый пример. Вот у нас есть сбалансированное дерево с некоторыми элементами. Мы вставляем в него число 32. После его добавления алгоритм проверяет баланс снизу вверх и обнаруживает, что у узла 15 фактор баланса стал равен -2 . Левая часть весит 1, а правая выросла до 3. Значит требуется произвести право-левый поворот, чтобы вернуть сбалансированность. Алгоритм берет правого потомка (узел 35) и делает вокруг него локальный правый поворот, выпрямляя структуру. Затем делается левый поворот уже вокруг самого узла 15. Узел 30 поднимается наверх и становится новым центром. Баланс восстановлен, и глубина дерева снова минимальна.

Слайд 6: Алгоритмическая сложность

Ради чего все эти сложности с поворотами? Благодаря строгой балансировке, алгоритмическая сложность поиска, вставки и удаления в AVL-дереве всегда составляет $O(\log n)$. И самое важное — это касается не только среднего, но и худшего случая. Никакие входные данные не смогут заставить AVL-дерево работать медленнее. Что касается памяти, то накладные расходы минимальны — нам требуется лишь $O(n)$ для хранения структуры и одно дополнительное поле в каждом узле для хранения его текущей высоты.

Слайд 7: Сравнение с конкурентами

Несмотря на все преимущества AVL-деревьев, в современном программировании, например в стандартной библиотеке C++, чаще используются красно-черные деревья. Это обосновано тем, что AVL-деревья поддерживают баланс гораздо строже, что не всегда играет на руку. Их максимальная высота всегда меньше, чем у красно-черных деревьев, поэтому поиск в AVL работает быстрее. Однако за эту строгость приходится платить: при интенсивном добавлении или удалении данных AVL-дерево будет всё чаще вызывать операции поворота. Поэтому это скорее инструмент с четкой специализацией.

Слайд 8: Где это применяется сегодня?

AVL-деревья идеально подходят для таких систем, где данных много, но читаются и ищутся они гораздо чаще, чем изменяются. Так, например, их применяют при создании in-методу индексов в базах данных. Также их используют в системном программном обеспечении и Real-time системах, где критически важно гарантировать стабильное и предсказуемое время отклика на запрос, исключая любые просадки производительности из-за неудачной структуры данных.

Слайд 9: Заключение

Подводя итог, AVL-дерево – это, я бы сказал, весьма гениальный математический концепт, который заложил основу для всех современных самобалансирующихся структур. В нём небольшие локальные затраты на поддержание порядка на этапе записи окупаются многократно гарантированной скоростью на этапе чтения. На этом у меня всё, спасибо за внимание. Буду рад ответить на ваши вопросы.